

到目前为止，我们介绍的并行计算模式都允许将计算每个输出元素的任务专门分配给或由一个线程拥有。因此，这些模式适用于所有者计算规则，其中每个线程可以在其指定的输出元素中写入，而不必担心其他线程的干扰。本章介绍了并行直方图计算模式，其中每个输出元素都可能被任何线程更新。因此，在更新输出元素时，必须协调各个线程，并避免任何可能损坏最终结果的干扰。实际上，还有许多其他重要的并行计算模式，其中无法轻易避免输出干扰。因此，并行直方图算法提供了在这些模式中发生输出干扰的一个例子。我们首先将研究一种基线方法，该方法使用原子操作来串行更新每个元素。这种基线方法简单但效率低下，通常会导致执行速度不佳。然后，我们将介绍一些广泛使用的优化技术，其中最重要的是私有化，可以显著提高执行速度同时保持正确性。这些技术的成本和效益取决于底层硬件以及输入数据的特性。因此，对于开发人员来说，了解这些技术的关键思想并能够在不同情况下进行推理是非常重要的。

9.1 Background

直方图是数据集中数据值出现次数或百分比的显示。在最常见的直方图形式中，数值间隔沿水平轴绘制，每个间隔中的数据值计数表示为从水平轴上升的矩形或柱状图的高度。例如，直方图可以用来展示短语“programming massively parallel processors”中字母的频率。为简单起见，我们假设输入短语全为小写。通过观察，我们可以看到字母“a”出现四次，“b”没有出现，“c”出现一次，以此类推。我们将每个数值间隔定义为连续的四个字母范围。因此，第一个数值间隔是“a”到“d”，第二个数值间隔是“e”到“h”，依此类推。图9.1显示了根据我们对数值间隔的定义显示短语“programming massively parallel processors”中字母频率的直方图。

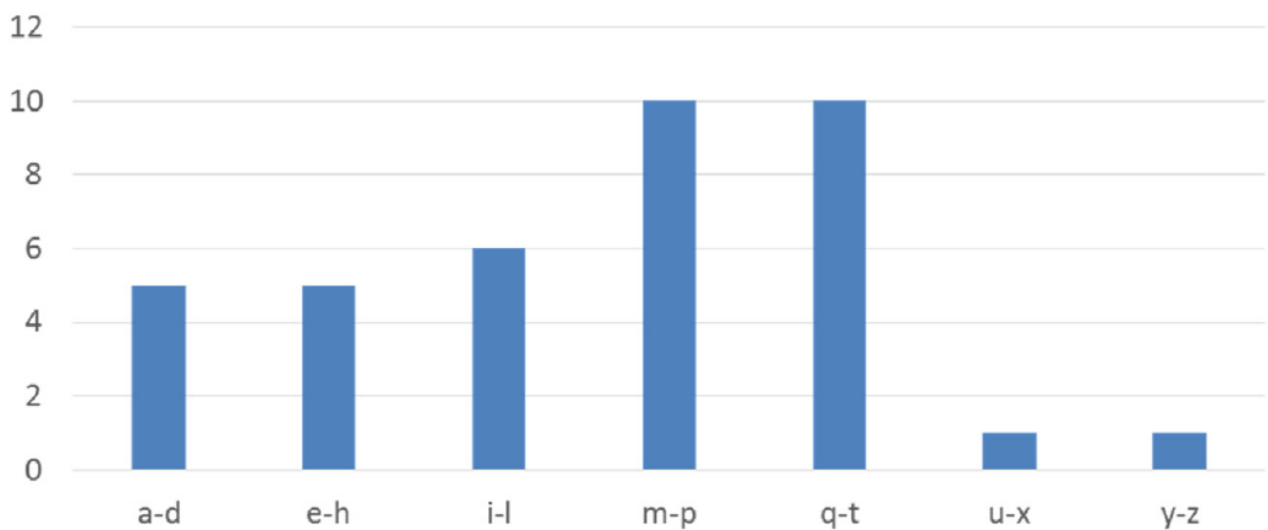


图9.1 短语“programming massively parallel processors”的直方图表示。

直方图提供了数据集的有用摘要。在我们的示例中，我们可以看到所表示的短语由字母组成，这些字母在字母表的中间间隔中非常集中，而在后续间隔中明显稀疏。直方图的形状有时被称为数据集的特征，并提供了一种快速确定数据集中是否存在重要现象的方法。例如，信用卡账户购买类别和位置的直方图形状可以用于检测欺诈使用。当直方图的形状与正常情况明显偏离时，系统会引发潜在关注的标志。

许多应用领域依赖直方图对数据集进行摘要分析。计算机视觉是其中之一。不同类型的对象图像的直方图，例如人脸与汽车，往往呈现不同的形状。例如，可以绘制图像或图像区域中像素亮度值的直方图。在晴天时，天空的这样一个直方图可能在亮度谱中的高值间隔中只有少量非常高的柱形。通过将图像划分为子区域并分析这些子区域的直方图，可以快速识别潜在包含感兴趣对象的图像的有趣子区域。计算图像子区域的直方图是计算机视觉中特征提取的重要方法，其中特征指的是图像中感兴趣的模式。实际上，每当有大量数据需要分析以提炼出有趣的事件时，直方图通常被用作基础计算。信用卡欺诈检测和计算机视觉显然符合这一描述。其他具有此类需求的应用领域包括语音识别、网站购买推荐以及天体物体运动相关的科学数据分析。

直方图可以以顺序方式轻松计算。图9.2显示了一个计算图9.1定义的直方图的顺序函数。为简单起见，直方图函数仅需识别小写字母。C代码假定输入数据集以char数组data的形式提供，并将直方图生成到int数组histo中（第01行）。输入数据项的数量在函数参数length中指定。for循环（第02至07行）顺序遍历数组，识别所访问位置data[i]中字符的字母索引，将字母索引保存到alphabet_position变量中，并增加与该间隔关联的histo[alphabet_position/4]元素。字母索引的计算依赖于输入字符串基于标准ASCII代码表示，其中字母“a”到“z”根据字母表中的顺序编码为连续值。

```
01 void histogram_sequential(char *data, unsigned int length,
    unsigned int *histo) {
02     for(unsigned int i = 0; i < length; ++i) {
03         int alphabet_position = data[i] - 'a';
04         if(alphabet_position >= 0 && alphabet_position < 26)
05             histo[alphabet_position/4]++;
06     }
07 }
08 }
```

图9.2 一个简单的C函数用于计算输入文本字符串的直方图。

尽管人们可能不知道每个字母的确切编码值，但可以假设字母的编码值是字母“a”的编码值加上该字母与“a”的字母表位置差。在输入中，每个字符都以其编码值存储。因此，表达式 data[i] - 'a'（第03行）可以得到字母的字母表位置，其中字母“a”的字母表位置为0。如果位置值大于或等于0且小于26，则数据字符确实是小写字母（第04行）。请记住，我们定义了间隔，使得每个间隔包含四个字母。因此，字母的间隔索引是其字母表位置值除以4。我们使用间隔索引来增加相应的histo数组元素（第05行）。

图9.2中的C代码非常简单且高效。算法的计算复杂度为 $O(N)$ ，其中N是输入数据元素的数量。在for循环中，数组元素按顺序访问，因此每当它们从系统DRAM中提取时，CPU缓存行都会得到很好的利用。histo数组非常小，可以很好地适应CPU的一级（L1）数据缓存，这确保了对histo元素的快速更新。对于大多数现代CPU，可以预期此代码的执行速度受到内存限制，即受限于将数据元素从DRAM载入CPU缓存的速度。

9.2 原子操作和基本直方图核心

将直方图计算并行化的最直接方法是启动与数据元素数量相同的线程，并让每个线程处理一个输入元素。每个线程读取其分配的输入元素，并增加相应字符的间隔计数器。图9.3展示了这种并行化策略的一个示例。请注意，多个线程需要更新相同的计数器（m-p），这是一种称为输出干扰(output interference)的冲突。程序员必须理解竞争条件和原子操作的概念，以便在其并行代码中自信地处理这种输出干扰。

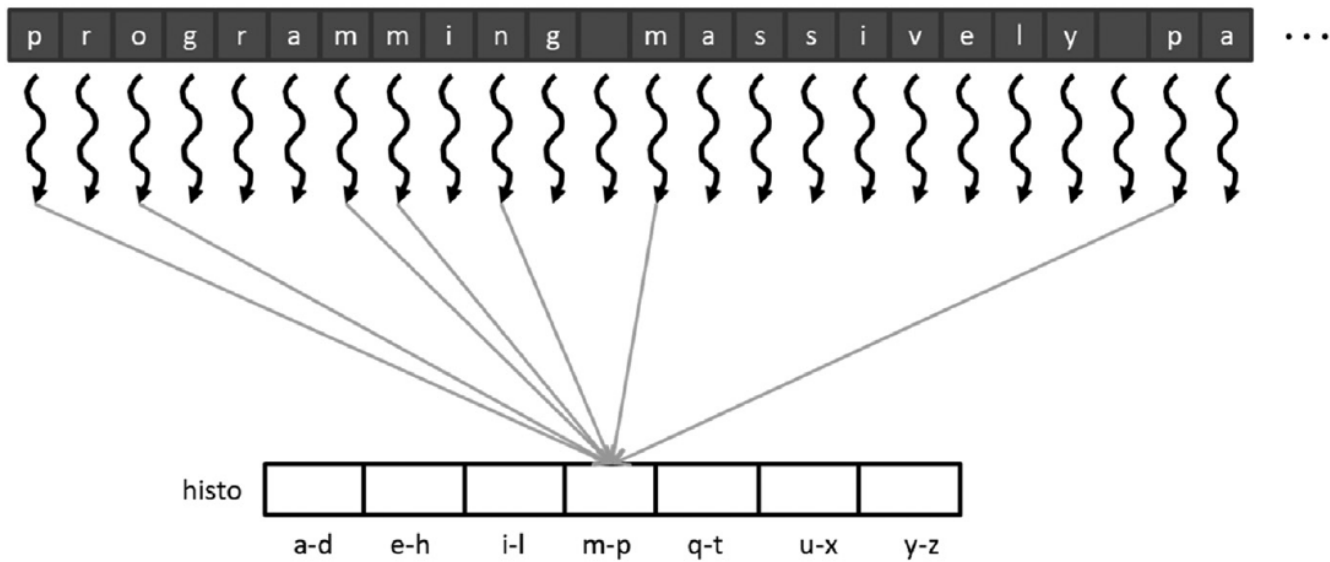


图9.3 基本直方图并行化方法。

对histo数组中间隔计数器的增量是对内存位置进行的更新，或者称为读-修改-写(Read-Modify-Write)操作。该操作涉及读取内存位置（读取），将原始值加一（修改），然后将新值写回内存位置（写入）。读-修改-写操作是协调协作活动经常使用的操作。

例如，当我们与航空公司进行航班预订时，我们会打开座位图并查找可用座位（读取），我们选择一个座位进行预订（修改），这会将座位图中该座位的状态更改为不可用（写入）。一个不好的潜在场景可能会发生如下：

- 两个顾客同时打开同一航班的座位图。
- 两个顾客都选择了同一个座位，比如9C。
- 两个顾客都将座位9C的状态更改为不可用。

在这个序列之后，两个顾客都认为他们有9C座位。我们可以想象，当他们登机后发现其中一个无法坐上预订的座位时，他们会遇到不愉快的情况！不管你信不信，由于航空公司预订软件的缺陷，这种不愉快的情况在现实生活中经常发生。

另一个例子是，一些商店允许顾客等待服务而不必排队。他们要求每位顾客从某个自助服务台取一个号码。有一个显示屏显示下一个将要接受服务的号码。当有服务代理可用时，代理要求顾客出示与号码匹配的票据，验证票据，并将显示号码更新为下一个更高的号码。理想情况下，所有顾客都将按照他们进入商店的顺序接受服务。一个不理想的结果是，两个顾客同时在两个自助服务台签到，并且都收到了相同号码的票据。当服务代理呼叫该号码时，两位顾客都希望自己是应该接受服务的人。

在这两个例子中，不良结果是由一种称为读-修改-写竞争条件的现象引起的，其中两个或更多并发更新操作的结果取决于所涉及操作的相对时间。一些结果是正确的，一些是错误的。图9.4展示了当两个线程尝试更新文本直方图示例中的相同histo元素时的竞争条件。图9.4中的每一行显示了一个时间段的活动，时间从上到下递进。

Time	Thread 1	Thread 2
1	(0) Old $\leftarrow$ histo[x]	
2	(1) New $\leftarrow$ Old + 1	
3	(1) histo[x] $\leftarrow$ New	
4		(1) Old $\leftarrow$ histo[x]
5		(2) New $\leftarrow$ Old + 1
6		(2) histo[x] $\leftarrow$ New

(A)

Time	Thread 1	Thread 2
1	(0) Old $\leftarrow$ histo[x]	
2	(1) New $\leftarrow$ Old + 1	
3		(0) Old $\leftarrow$ histo[x]
4	(1) histo[x] $\leftarrow$ New	
5		(1) New $\leftarrow$ Old + 1
6		(1) histo[x] $\leftarrow$ New

(B)

图9.4 直方图数组元素更新中的竞争条件： (A) 指令的一个可能的交错排列； (B) 指令的另一个可能的交错排列。

图9.4A描述了一种情况，在这种情况下，线程1在时间段1至3期间完成了其读-修改-写序列的所有三个部分，然后线程2在时间段4开始其序列。括号中每个操作前面的值显示了写入目标的值，假设histo[x]的初始值为0。在这种情况下，histo[x]之后的值是2，正如人们所期望的那样。也就是说，两个线程成功地增加了histo[x]。元素值从0开始，在操作完成后变为2。

在图9.4B中，两个线程的读-修改-写序列是重叠的。请注意，线程1在时间段4将新值写入histo[x]。当线程2在时间段3读取histo[x]时，它仍然是值为0。因此，它计算的新值并最终写入histo[x]的值是1而不是2。问题在于，线程2在线程1完成更新之前就读取了histo[x]。最终结果是，histo[x]之后的值是1，这是不正确的。线程1的更新被丢失。

Time	Thread 1	Thread 2
1		(0) Old $\leftarrow$ histo[x]
2		(1) New $\leftarrow$ Old + 1
3		(1) histo[x] $\leftarrow$ New
4	(1) Old $\leftarrow$ histo[x]	
5	(2) New $\leftarrow$ Old + 1	
6	(2) histo[x] $\leftarrow$ New	

(A)

Time	Thread 1	Thread 2
1		(0) Old $\leftarrow$ histo[x]
2		(1) New $\leftarrow$ Old + 1
3	(0) Old $\leftarrow$ histo[x]	
4		(1) histo[x] $\leftarrow$ New
5	(1) New $\leftarrow$ Old + 1	
6	(1) histo[x] $\leftarrow$ New	

(B)

图9.5 线程2在线程1之前运行的竞争条件场景： (A) 一种可能的指令交错排列； (B) 另一种可能的指令交错排列。

在并行执行中，线程可以以任何相对顺序运行。在我们的示例中，线程2可以很容易地在线程1之前开始其更新序列。图9.5显示了两种这样的情况。在图9.5A中，线程2在线程1开始之前完成了更新。在图9.5B中，线程1在线程2完成之前开始了更新。很明显，在图9.5A中的序列产生了histo[x]的正确结果，而在图9.5B中的序列产生了不正确的结果。

最终的histo[x]值取决于所涉及操作的相对时间，这表明存在竞争条件。我们可以通过消除线程1和线程2操作序列的可能交错来消除这种变化。也就是说，我们希望允许图9.4A和9.5A中显示的时间而消除图9.4B和9.5B中显示的可能性。这可以通过使用原子操作来实现。

内存位置上的原子操作是指对该内存位置执行读-修改-写序列的操作，以使没有其他读-修改-写序列可以与之重叠。也就是说，操作的读取、修改和写入部分形成一个不可分割的单元，因此称为原子操作。实际上，原子操作是通过硬件支持来实现的，以阻止其他操作对相同位置的访问，直到当前操作完成为止。在我们的示例中，这种支持消除了图9.4B和9.5B中所示的可能性，因为后续线程在前导线程完成其操作之前无法开始其更新序列。

重要的是要记住，原子操作不会强制执行任何特定的线程执行顺序。在我们的示例中，原子操作允许图9.4A和9.5A中显示的两种顺序。线程1可以在线程2之前或之后运行。正在执行的规则是，如果两个线程对相同的内存位置执行原子操作，则后续线程执行的原子操作在前导线程的原子操作完成之前不能开始。这有效地对在内存位置上执行的原子操作进行了串行化。

原子操作通常根据对内存位置进行的修改来命名。在我们的文本直方图示例中，我们正在向内存位置添加一个值，因此该原子操作称为原子加法。其他类型的原子操作包括减法、增加、减少、最小值、最大值、逻辑与和逻辑或。CUDA内核可以通过函数调用在内存位置上执行原子加法操作。

```
int atomicAdd(int* address, int val);
```

`atomicAdd`函数是一个内置函数（参见侧边栏“内置函数”），它被编译成硬件原子操作指令。这个指令读取全局或共享内存中由地址参数指向的32位字，将`val`加到旧内容上，并将结果存储回内存的同一地址。函数返回地址处的旧值。

内置函数

现代处理器通常提供特殊指令，这些指令要么执行关键功能（如原子操作），要么大幅提升性能（如矢量指令）。这些指令通常被暴露给程序员作为内置函数，或简称为内置函数。

从程序员的角度来看，这些函数是库函数。然而，编译器对它们进行特殊处理；每次这样的调用都会被翻译成相应的特殊指令。最终代码中通常没有函数调用，只有与用户代码一致的特殊指令。所有主流的现代编译器，如GNU编译器集合（`gcc`）、英特尔C编译器和Clang/LLVM C编译器都支持内置函数。

```
01 __global__ void histo_kernel(char *data, unsigned int length,
02                               unsigned int *histo) {
03     unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
04     if (i < length) {
05         int alphabet_position = data[i] - 'a';
06         if (alphabet_position >= 0 && alphabet_position < 26) {
07             atomicAdd(&(histo[alphabet_position/4]), 1);
08         }
09     }
```

图9.6 一个用于计算直方图的CUDA内核。

图9.6展示了一个执行并行直方图计算的CUDA内核。这段代码与图9.2中的顺序代码相似，但有两个关键区别。第一个区别是将对输入元素的循环替换为线程索引计算（第2行）和边界检查（第3行），以将一个线程分配给每个输入元素。第二个区别是在图9.2中的增量表达式：

```
histo[alphabet_position/4]++
```

在图9.6中变成了`atomicAdd()`函数调用（第6行）。要更新的位置的地址，即`&(histo[alphabet_position/4])`，是第一个参数。要添加到该位置的值，即1，是第二个参数。这样可以确保不同线程对任何`histo`数组元素的同时更新都得到正确的串行化处理。

9.3 原子操作的延迟和吞吐量

图9.6内核中使用的原子操作确保了对位置的同时更新的正确性。我们知道，在并程序的任何部分进行串行化可能会显著增加执行时间并降低程序的执行速度。因此，重要的是这样的串行化操作占用尽可能少的执行时

间。

正如我们在第5章《内存体系结构和数据局部性》中学到的那样，DRAM中数据的访问延迟可能需要数百个时钟周期。在第4章《计算体系结构和调度》中，我们了解到GPU使用零周期上下文切换来容忍这种延迟。在第6章《性能考虑》中，我们了解到只要我们有许多线程，它们的内存访问延迟可以重叠，执行速度就受到内存系统吞吐量的限制。因此，重要的是GPU充分利用DRAM突发、通道和通道来实现高内存访问吞吐量。

读者现在应该清楚，实现高内存访问吞吐量的关键是同时进行许多DRAM访问。不幸的是，当许多原子操作更新相同的内存位置时，这种策略就会崩溃。在这种情况下，一个后续线程的读-修改-写序列在前导线程的读-修改-写序列完成之前不能开始。正如图9.7所示，相同内存位置的原子操作的执行只能有一个在进行中。每个原子操作的持续时间大约是一个内存加载的延迟（原子操作时间的左侧部分）加上一个内存存储的延迟（原子操作时间的右侧部分）。每个读-修改-写操作的这些时间部分的长度通常为数百个时钟周期，这定义了必须为每个原子操作服务的最小时间量，并限制了吞吐量，即可以执行原子操作的速率。

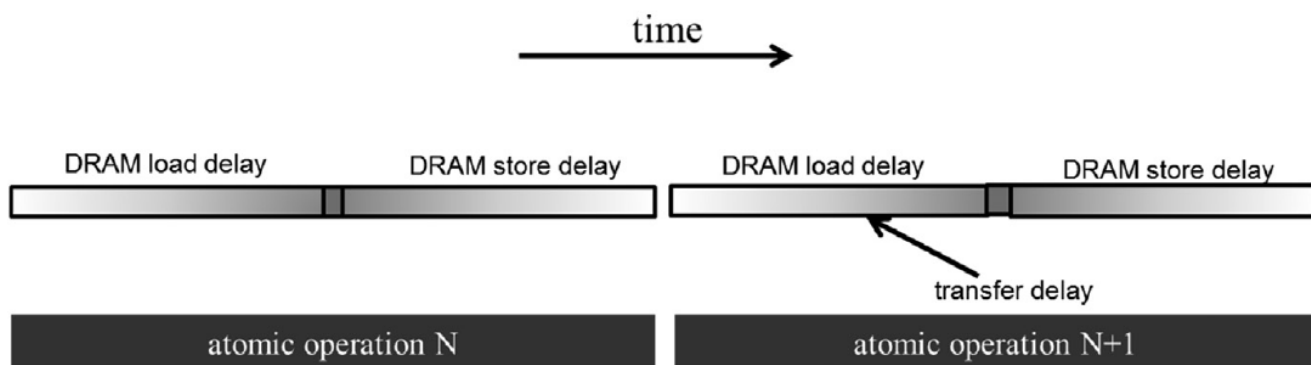


图9.7 原子操作的吞吐量由内存访问延迟决定。

例如，假设一个内存系统有一个64位（8字节）双数据率DRAM接口每个通道，八个通道，1 GHz时钟频率，典型的访问延迟为200个周期。内存系统的峰值访问吞吐量为 $8 \text{ (字节/传输)} \times 2 \text{ (每个通道每个时钟的传输)} \times 1 \text{ G (每秒的时钟数)} \times 8 \text{ (通道)} = 128 \text{ GB/s}$ 。假设每个访问的数据为4字节，则系统的峰值访问吞吐量为每秒32 G数据元素。

然而，在对特定内存位置执行原子操作时，可以实现的最高吞吐量是每400个周期执行一个原子操作（200个周期用于读取，200个周期用于写入）。这意味着基于时间的吞吐量为 $1/400 \text{ 个原子/时钟} \times 1 \text{ G (每秒的时钟)} = 2.5 \text{ M个原子/秒}$ 。这比大多数用户对GPU内存系统的预期要低得多。此外，原子操作序列的长延迟可能会主导内核执行时间，并极大降低内核的执行速度。

在实践中，并非所有的原子操作都会在单个内存位置上执行。在我们的文本直方图示例中，直方图有七个间隔。如果输入字符在字母表中均匀分布，那么原子操作将均匀分布在直方图元素中。这将使吞吐量提高到 $7 \times 2.5 \text{ M} = 17.5 \text{ M个原子操作每秒}$ 。实际上，增加的因素往往比直方图中的间隔数要低得多，因为字符在字母表中往往具有偏向分布。例如，在图9.1中，我们看到示例短语中的字符在m-p和q-t间隔中有很大的偏向性。更新这些间隔的大量竞争流量可能会将可实现的吞吐量降低到大约 $(28/10) \times 2.5 \text{ M} = 7 \text{ M}$ 。

提高原子操作的吞吐量的一种方法是减少对高度争用位置的访问延迟。缓存存储器是减少内存访问延迟的主要工具。因此，现代GPU允许在最后一级缓存中执行原子操作，该缓存在所有流多处理器（SM）之间共享。在原子操作期间，如果在最后一级缓存中找到要更新的变量，则会更新缓存中的变量。如果在最后一级缓存中找不到它，则会触发缓存未命中，并将其带入缓存，然后在缓存中更新。由于由原子操作更新的变量往往被许多线程密集访问，所以这些变量一旦从DRAM中带入缓存就会保留在缓存中。由于访问最后一级缓存的时间在几十个周期而不是数百个周期，所以与GPU的早期一代相比，原子操作的吞吐量至少提高了一个数量级。这是现代GPU大多数支持在最后一级缓存中执行原子操作的重要原因。

9.4 私有化

改善原子操作吞吐量的另一种方法是通过将流量从高度竞争的位置导向其他位置来缓解争用，这可以通过一种称为私有化(privatization)的技术来实现，这种技术通常用于解决并行计算中的严重输出干扰问题。其思想是将高度竞争的输出数据结构复制到私有副本中，以便每个线程子集可以更新其私有副本。好处在于，私有副本可以在较少的争用和通常更低的延迟下进行访问。这些私有副本可以显著增加更新数据结构的吞吐量。缺点是，私有副本需要在计算完成后合并到原始数据结构中。必须仔细权衡争用水平和合并成本。因此，在大规模并行系统中，通常对线程子集而不是单个线程进行私有化。

在我们的文本直方图示例中，我们可以创建多个私有直方图，并指定一部分线程来更新每个直方图。例如，我们可以创建两个私有副本，让偶数索引的块更新其中一个，奇数索引的块更新另一个。另一个例子是，我们可以创建四个私有副本，并让具有形式 $4n+i$ 的块的索引更新第 i 个私有版本，其中 i 取 0 到 3。一个常见的方法是为每个线程块创建一个私有副本。这种方法有多个后面将看到的优点。

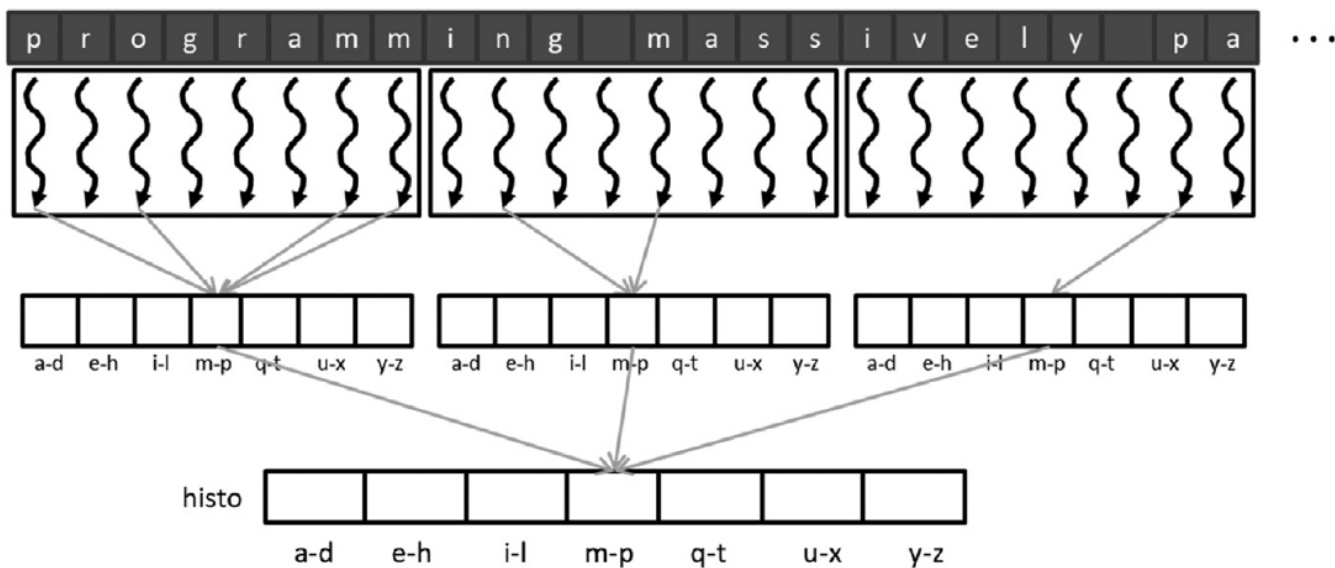


图 9.8 私有直方图副本减少了原子操作的竞争。

图 9.8 展示了私有化如何应用到图 9.3 的文本直方图示例中。在这个示例中，线程被组织成线程块，每个线程块包含八个线程（实际上，线程块要大得多）。每个线程块接收一个私有直方图副本，并对其进行更新。如图 9.8 所示，与所有更新相同直方图条的所有线程之间的竞争不同，竞争只会发生在同一块内的线程之间，以及在最后合并私有副本时。

```

01  __global__ void histo_private_kernel(char *data, unsigned int length,
                                unsigned int *histo) {
02      unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
03      if(i < length) {
04          int alphabet_position = data[i] - 'a';
05          if (alphabet_position >= 0 && alphabet_position < 26) {
06              atomicAdd(&(histo[blockIdx.x*NUM_BINS + alphabet_position/4]), 1);
07          }
08      }
09      if(blockIdx.x > 0) {
10          __syncthreads();
11          for(unsigned int bin=threadIdx.x; bin<NUM_BINS; bin += blockDim.x){
12              unsigned int binValue = histo[blockIdx.x*NUM_BINS + bin];
13              if(binValue > 0) {
14                  atomicAdd(&(histo[bin]), binValue);
15              }
16          }
17      }
18  }

```

图 9.9 直方图内核，使用全局内存中的线程块的私有版本。

图 9.9 展示了一个简单的内核，它为每个块创建并关联一个私有副本的直方图。在这种方案中，最多 1024 个线程将在直方图的副本上工作。在这个内核中，私有直方图位于全局内存中。这些私有副本很可能被缓存在 L2 缓存中，以减少延迟并提高吞吐量。

图 9.9 内核的第一部分（行 02 到 08）与图 9.6 中的内核类似，但有一个关键区别。图 9.9 中的内核假定主机代码将为 `histo` 数组分配足够的设备内存来容纳所有私有副本的直方图，这相当于 `gridDim.x * NUM_BINS * 4` 字节。这体现在行 06 中，每个线程在对直方图元素（直方图的柱状图）进行原子添加时，将 `blockIdx.x * NUM_BINS` 的偏移量添加到索引中。这个偏移量将位置移动到线程所属块的私有副本。在这种情况下，争用水平会减少一个因子，这个因子大约等于所有 SM 中活动块的数量。减少争用的效果可能会导致内核更新吞吐量的数量级的改进。

在执行结束时，每个线程块将私有副本中的值提交到由块 0 生成的版本中（第 09-17 行）。也就是说，我们将块 0 的私有副本提升为公共副本，该副本将保存所有块产生的总结果。线程首先等待块内的其他线程完成对私有副本的更新（第 10 行）。接下来，线程遍历私有直方图条目（第 11 行），每个线程负责提交一个或多个私有条目。使用循环来适应任意数量的条目。每个线程读取其负责的私有条目的值（第 13 行），并检查该条目是否非零（第 13 行）。如果是非零值，线程将通过原子方式将其添加到块 0 的副本中（第 14 行）。请注意，添加操作需要以原子方式执行，因为来自多个块的线程可以同时相同位置执行添加操作。因此，在内核执行结束时，最终的直方图将位于 `histo` 数组的前 `NUM_BINS` 元素中。由于每个块的只有一个线程将在内核执行的这个阶段更新任何给定的 `histo` 数组元素，因此每个位置的争用水平非常适中。

将直方图的私有副本按线程块进行创建的一个好处是，线程可以使用 `__syncthreads()` 在提交之前等待彼此。如果多个块访问私有副本，我们将需要调用另一个内核来合并私有副本，或者使用其他复杂的技术。另一个在线程块基础上创建直方图的私有副本的好处是，如果直方图中的条目数足够少，则可以将直方图的私有副本声明为共享内存中。如果私有副本由多个块访问，将无法使用共享内存，因为块无法访问彼此的共享内存。

请记住，内存访问的延迟减少直接导致在相同内存位置上的原子操作的吞吐量提高。通过将数据放入共享内存，可以大幅减少访问内存的延迟。共享内存对每个 SM 都是私有的，访问延迟非常短（几个周期）。这种降低的延迟直接转化为原子操作的增加吞吐量。


```

01 __global__ void histo_private_kernel(char* data, unsigned int length,
                                     unsigned int* histo) {
02     // Initialize privatized bins
03     __shared__ unsigned int histo_s[NUM_BINS];
04     for(unsigned int bin = threadIdx.x; bin< NUM_BINS; bin += blockDim.x) {
05         histo_s[bin] = 0u;
06     }
07     __syncthreads();
08     // Histogram
09     unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
10     if(i < length) {
11         int alphabet_position = data[i] - 'a';
12         if(alphabet_position >= 0 && alphabet_position < 26) {
13             atomicAdd(&(histo_s[alphabet_position/4]), 1);
14         }
15     }
16     __syncthreads();
17     // Commit to global memory
18     for(unsigned int bin=threadIdx.x; bin<NUM_BINS; bin += blockDim.x) {
19         unsigned int binValue = histo_s[bin];
20         if(binValue > 0) {
21             atomicAdd(&(histo[bin]), binValue);
22         }
23     }
24 }

```

图9.10 私有化的文本直方图内核使用共享内存。

图9.10展示了一个将私有副本存储在共享内存中而不是全局内存中的私有化直方图内核。与图 9.9 中内核代码的关键区别在于，直方图的私有副本在共享内存中分配，在 `histo_s` 数组中初始化为 0，并由块的线程并行执行（第 02-06 行）。屏障同步（第 07 行）确保在任何线程开始更新它们之前，私有直方图的所有条目都已经正确初始化。剩下的代码与图 9.9 中的代码完全相同，只是第一个原子操作是在共享内存数组 `histo_s` 的元素上执行的（第 13 行），私有条目的值稍后从那里读取（第 19 行）。

9.5 粗化

我们已经看到，私有化在减少原子操作的竞争方面是有效的，并且将私有化的直方图存储在共享内存中可以降低每个原子操作的延迟。然而，私有化的开销是需要将私有副本提交到公共副本。这个提交操作是每个线程块执行一次的。因此，我们使用的线程块越多，这个开销就越大。当线程块并行执行时，这个开销通常是值得的。然而，如果启动的线程块数量超过硬件同时执行的数量，硬件调度将串行化这些线程块。在这种情况下，私有化的开销是不必要的。

我们可以通过线程粗化来减少私有化的开销。换句话说，我们可以通过减少线程块的数量并使每个线程处理多个输入元素来减少需要提交到公共副本的私有副本的数量。在本节中，我们将研究两种为线程分配多个输入元素的策略：连续分区和交错分区。

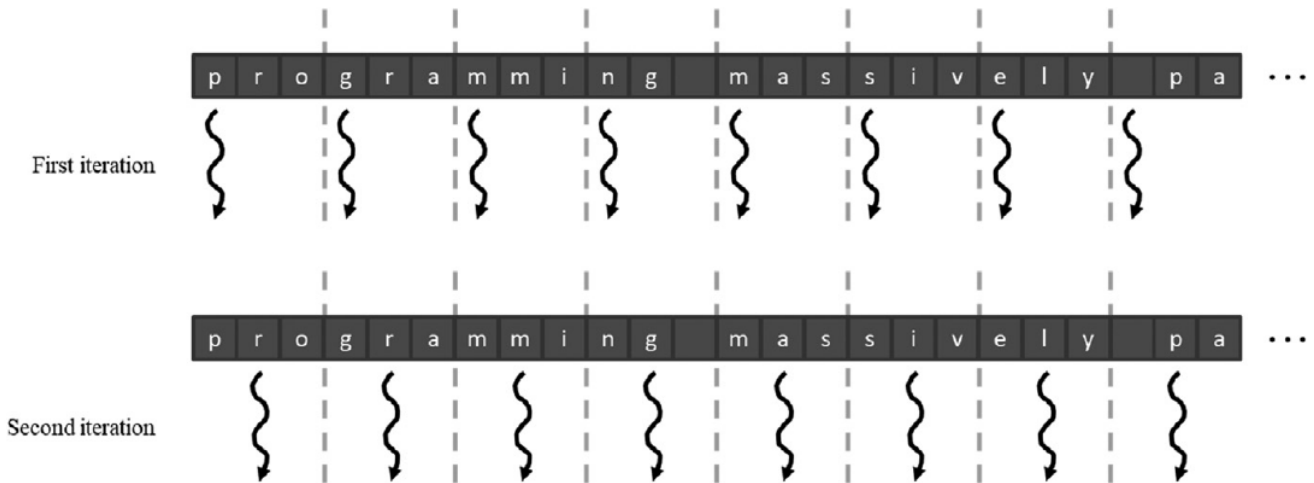


图9.11 连续分区的输入元素。

```

01  __global__ void histo_private_kernel(char* data, unsigned int length,
                                     unsigned int* histo) {
02      // Initialize privatized bins
03      __shared__ unsigned int histo_s[NUM_BINS];
04      for(unsigned int bin = threadIdx.x; bin<NUM_BINS; bin += blockDim.x) {
05          histo_s[binIdx] = 0u;
06      }
07      __syncthreads();
08      // Histogram
09      unsigned int tid = blockIdx.x*blockDim.x + threadIdx.x;
10      for(unsigned int i=tid*CFACTOR; i<min((tid+1)*CFACTOR, length); ++i) {
11          int alphabet_position = data[i] - 'a';
12          if(alphabet_position >= 0 && alphabet_position < 26) {
13              atomicAdd(&(histo_s[alphabet_position/4]), 1);
14          }
15      }
16      __syncthreads();
17      // Commit to global memory
18      for(unsigned int bin = threadIdx.x; bin<NUM_BINS; bin += blockDim.x) {
19          unsigned int binValue = histo_s[binIdx];
20          if(binValue > 0) {
21              atomicAdd(&(histo[binIdx]), binValue);
22          }
23      }
24  }

```

图9.12 直方图内核使用连续分区进行粗化。

图9.11展示了连续分区策略的示例。输入被分成连续的片段，每个片段分配给一个线程。图9.12展示了应用了连续分区策略的直方图内核的粗化。与图9.10的区别在于线09-10。在图9.10中，输入元素索引 i 对应于全局线程索引，因此每个线程接收一个输入元素。在图9.11中，输入元素索引是一个循环的索引，该循环从 $\text{tid} * \text{CFACTOR}$ 到 $(\text{tid} + 1) * \text{CFACTOR}$ 迭代，其中 CFACTOR 是粗化因子。因此，每个线程获取 CFACTOR 元素的连续段。循环边界中的 min 操作确保末尾的线程不会越界读取。

将数据分区到连续的片段中在概念上是简单直观的。在CPU上，其中并行执行通常涉及少量线程时，连续分区通常是性能最好的策略，因为每个线程的顺序访问模式可以充分利用缓存行。由于每个CPU缓存通常只支持少量线程，因此不同线程之间的缓存使用干扰很少。一旦为一个线程带入了缓存行中的数据，可以预期该数据将在随后的访问中保持在那里。

相反，在GPU上进行连续分区会导致次优的内存访问模式。正如我们在第5章，内存体系结构和数据局部性中所学到的，一个SM中同时活跃的大量线程通常会导致缓存中有太多的干扰，以至于一个单线程不能期望数据在缓

存中保持以供单线程的所有顺序访问使用。相反，我们需要确保warp中的线程访问连续的位置以启用内存合并。这个观察结果激发了交错分区。

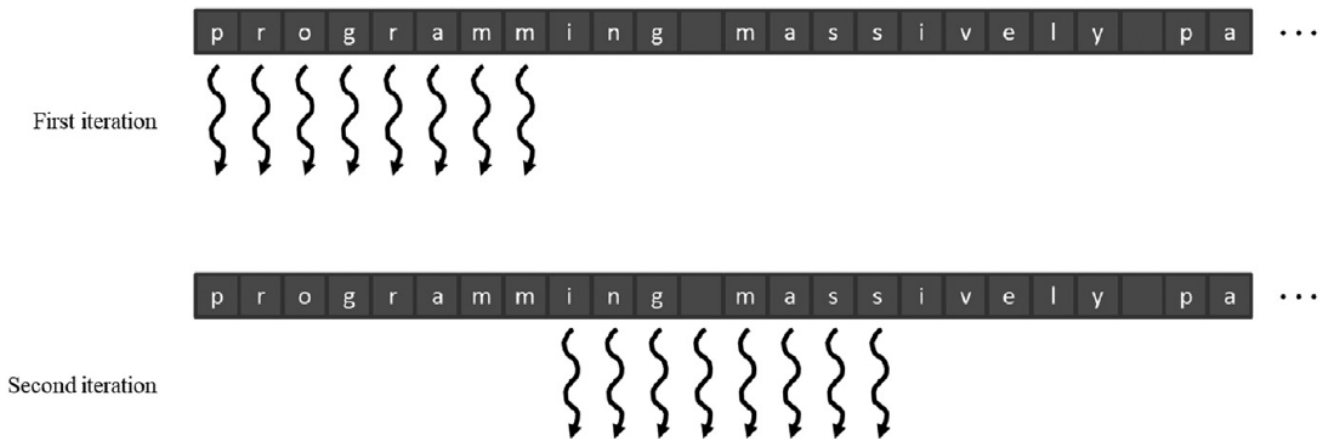


图9.13 交错分区的输入元素。

图9.13展示了交错分区策略的示例。在第一次迭代中，八个线程访问字符0到7（“programm”）。通过内存合并，所有元素将只需一次DRAM访问即可获取。在第二次迭代中，四个线程在一个合并的内存访问中访问字符“ing mass”。这应该清楚地解释了为什么这被称为交错分区：不同线程要处理的分区是相互交错的。显然，这是一个玩具示例，在现实中，会有更多的线程。还有更加微妙的性能考虑。例如，每个线程应在每次迭代中处理四个字符（一个32位字），以充分利用缓存和SM之间的互连带宽。

```

01  __global__ void histo_private_kernel(char* data, unsigned int length,
                                     unsigned int* histo){
02      // Initialize privatized bins
03      __shared__ unsigned int histo_s[NUM_BINS];
04      for(unsigned int bin=threadIdx.x; bin<NUM_BINS; bin += blockDim.x) {
05          histo_s[binIdx] = 0u;
06      }
07      __syncthreads();
08      // Histogram
09      unsigned int tid = blockIdx.x*blockDim.x + threadIdx.x;
10      for(unsigned int i = tid; i < length; i += blockDim.x*gridDim.x) {
11          int alphabet_position = data[i] - 'a';
12          if(alphabet_position >= 0 && alphabet_position < 26) {
13              atomicAdd(&(histo_s[alphabet_position/4]), 1);
14          }
15      }
16      __syncthreads();
17      // Commit to global memory
18      for(unsigned int bin = threadIdx.x; bin<NUM_BINS; bin += blockDim.x) {
19          unsigned int binValue = histo_s[binIdx];
20          if(binValue > 0) {
21              atomicAdd(&(histo[binIdx]), binValue);
22          }
23      }
24  }

```

图9.14 直方图内核使用交错分区进行粗化。

图9.14展示了应用了交错分区策略的直方图内核的粗化。与图9.10和9.12的区别再次在线09-10。在循环的第一次迭代中，每个线程使用其全局线程索引访问数据数组：线程0访问元素0，线程1访问元素1，线程2访问元素2，依此类推。因此，所有线程共同处理输入的第一个blockDim.x * gridDim.x元素。在第二次迭代中，所有线程将blockDim.x * gridDim.x添加到它们的索引，并共同处理下一个blockDim.x * gridDim.x元素的部分。

当一个线程的索引超出输入缓冲区的有效范围时（其私有`i`变量值大于或等于`length`），线程已经完成了对其分区的处理，并将退出循环。因为缓冲区的大小可能不是线程总数的倍数，所以一些线程可能不会参与最后一个部分的处理。因此，一些线程将比其他线程执行一个更少的循环迭代。

9.6 聚合

一些数据集在局部区域有大量相同的数据值。例如，在天空照片中，可能存在大片像素值相同的区域。这种大量相同值的高集中度会导致严重的竞争和并行直方图计算的吞吐量降低。

针对这种数据集，一种简单而有效的优化方法是让每个线程将连续的更新聚合成单个更新，如果它们更新直方图的不同元素（Merrill, 2015）。这种聚合可以减少高度争议的直方图元素的原子操作数量，从而提高计算的有效吞吐量。

```

01  __global__ void histo_private_kernel(char* data, unsigned int length,
                                     unsigned int* histo){
02      // Initialize privatized bins
03      __shared__ unsigned int histo_s[NUM_BINS];
04      for(unsigned int bin = threadIdx.x; bin < NUM_BINS; bin += blockDim.x){
05          histo_s[bin] = 0u;
06      }
07      __syncthreads();
08      // Histogram
09      unsigned int accumulator = 0;
10      int prevBinIdx = -1;
11      unsigned int tid = blockIdx.x*blockDim.x + threadIdx.x;
12      for(unsigned int i = tid; i < length; i += blockDim.x*gridDim.x) {
13          int alphabet_position = data[i] - 'a';
14          if(alphabet_position >= 0 && alphabet_position < 26) {
15              int bin = alphabet_position/4;
16              if(bin == prevBinIdx) {
17                  ++accumulator;
18              } else {
19                  if(accumulator > 0) {
20                      atomicAdd(&(histo_s[prevBinIdx]), accumulator);
21                  }
22                  accumulator = 1;
23                  prevBinIdx = bin;
24              }
25          }
26      }
27      if(accumulator > 0) {
28          atomicAdd(&(histo_s[prevBinIdx]), accumulator);
29      }
30      __syncthreads();
31      // Commit to global memory
32      for(unsigned int bin = threadIdx.x; bin < NUM_BINS; bin += blockDim.x) {
33          unsigned int binValue = histo_s[bin];
34          if(binValue > 0) {
35              atomicAdd(&(histo[bin]), binValue);
36          }
37      }
38  }

```

图9.15 一个聚合的文本直方图内核。

图9.15展示了一个聚合文本直方图内核。与图9.14中的内核相比，主要变化如下：每个线程声明了一个额外的累加器变量（第09行），用于跟踪到目前为止已聚合的更新数量，以及一个`prevBinIdx`变量（第10行），用于

跟踪上次遇到并正在聚合的直方图元素的索引。每个线程将累加器变量初始化为零，表示尚未开始聚合更新，并将`prevBinIdx`初始化为21，以使任何字母输入都不匹配它。

当找到一个字母数据时，线程比较要更新的直方图元素的索引与正在聚合的元素的索引（第16行）。如果索引相同，线程简单地增加累加器（第17行），将聚合更新的连续更新扩展一个。如果索引不同，表示聚合更新到直方图元素的连续更新已结束。线程使用原子操作将累加器值添加到由`prevBinIdx`跟踪的直方图元素（第19-21行）。这有效地清除了先前连续聚合更新的总和。

使用这种方案，更新总是至少落后一个元素。在极端情况下，如果没有连续更新，所有更新将始终落后一个元素。这就是为什么在一个线程完成扫描所有输入元素并退出循环后，线程需要检查是否需要清除累加器值（第27行）的原因。如果需要，累加器值将被清除到正确的`histo_s`元素（第28行）。

一个重要的观察是，聚合内核需要更多的语句和变量。因此，如果争用率低，聚合内核可能比简单内核执行速度慢。然而，如果数据分布导致原子操作执行的严重竞争，聚合可能会导致速度显著提高。增加的if语句可能会出现控制分歧。但是，如果没有竞争或者竞争很激烈，由于线程要么都清除累加器值，要么都在一个连续更新中，因此控制分歧很小。在某些线程在一个连续更新中，而其他线程正在清除它们的累加器值的情况下，控制分歧可能会被减少的竞争所抵消。

9.7 总结

直方图计算对于分析大型数据集非常重要。它还代表了一类重要的并行计算模式，其中每个线程的输出位置依赖于数据，这使得应用所有者计算规则变得不可行。因此，它是引入读取-修改-写入竞争条件概念和确保并发读取-修改-写入操作到同一内存位置完整性的原子操作的自然载体。

不幸的是，正如我们在本章中所解释的，原子操作的吞吐量比简单的内存读取或写入操作低得多，因为它们的吞吐量大约是内存延迟的两倍的倒数。因此，在存在严重争用的情况下，直方图计算的计算吞吐量可能会非常低。私有化被引入作为一种重要的优化技术，它系统地减少了争用，并进一步可以启用共享内存的使用，共享内存支持低延迟，因此具有高吞吐量。实际上，支持块内线程之间快速原子操作是共享内存的一个重要用例。粗化也被应用来减少需要合并的私有副本数量，并且比较了使用连续划分和交替划分的不同粗化策略。最后，对于引起严重争用的数据集，聚合也可以导致显着更高的执行速度。